

Section A: Concept Application

1. How relational databases help maintain accuracy in expense records?

Relational databases maintain accuracy using primary keys, foreign keys, constraints, and relationships between tables. They prevent duplicate and invalid data.

2. Why are constraints important in personal finance data?

Constraints ensure data integrity by preventing invalid entries such as negative amounts, duplicate IDs, or incorrect references.

3. How does GROUP BY help analyze spending patterns?

GROUP BY groups records based on categories, users, or dates, making it easy to calculate total spending and identify spending trends.

4. Explain a scenario where rollback is required during expense entry.

If a user accidentally enters ₹5000 instead of ₹500 and notices the mistake before committing, a ROLLBACK can undo the incorrect transaction.

5. How do views help users track monthly expenses efficiently?

Views store frequently used queries, allowing users to quickly access summarized expense information without writing complex queries repeatedly.

6. Why use triggers for automatic category or balance updates?

Triggers automatically execute actions when data changes, ensuring balances and categories remain updated without manual intervention.

Section B: SQL Hands-On

1. DDL Understanding

Why are Foreign Keys used?

Foreign keys maintain referential integrity. Every user_id in the expenses table must exist in the users table, preventing invalid references.

Issue if Foreign Keys are removed

Orphan records may occur. For example, an expense record may reference a deleted user, causing inconsistent data.

2. DML Operations

(3,'Entertainment');

Insert 10 Expense Records

SQL Insert 5 Users

SQL

INSERT INTO users VALUES

(1,'Rahul','rahul@gmail.com','2025-01-01'),

(2,'Amit','amit@gmail.com','2025-01-02'),

(3,'Priya','priya@gmail.com','2025-01-03'),

(4,'Neha','neha@gmail.com','2025-01-04'),

(5,'Karan','karan@gmail.com','2025-01-05');

Insert 3 Categories

SQL

INSERT INTO categories VALUES

(1,'Food'),

(2,'Rent'),

INSERT INTO expenses VALUES

(101,1,1,500,'2025-06-01'),

(102,1,2,8000,'2025-06-02'),

(103,2,1,300,'2025-06-03'),

(104,2,3,700,'2025-06-04'),

(105,3,2,7500,'2025-06-05'),

(106,3,1,400,'2025-06-06'),

(107,4,3,1200,'2025-06-07'),

(108,4,1,350,'2025-06-08'),

(109,5,2,9000,'2025-06-09'),

(110,5,3,600,'2025-06-10');

Update One Incorrect Expense

SQL

UPDATE expenses

SET amount = 550

WHERE expense_id = 101;

Delete One Expense

SQL

DELETE FROM expenses

WHERE amount < 400;

3. Data Retrieval

Display All Expenses with Details

SQL

SELECT

e.expense_date,

e.amount,

u.name,

c.category_name

FROM expenses e

INNER JOIN users u

ON e.user_id = u.user_id

INNER JOIN categories c

ON e.category_id = c.category_id;

Total Expense Amount Per Category

SQL

SELECT

```
c.category_name,  
SUM(e.amount) AS total_expense  
FROM expenses e  
INNER JOIN categories c  
ON e.category_id = c.category_id  
GROUP BY c.category_name;
```

Users Sorted by Total Spending

SQL

```
SELECT  
u.name,  
SUM(e.amount) AS total_spending  
FROM users u  
INNER JOIN expenses e  
ON u.user_id = e.user_id  
GROUP BY u.name  
ORDER BY total_spending DESC;
```

4. Views

Create ActiveUsersView

SQL

```
CREATE VIEW ActiveUsersView AS
```

```
SELECT
u.name,
u.email
FROM users u
INNER JOIN expenses e
ON u.user_id = e.user_id
GROUP BY u.user_id,u.name,u.email
HAVING COUNT(e.expense_id) > 5;
```

Query the View

SQL

```
SELECT * FROM ActiveUsersView;
```

Section C: Mini Project

CRUD Queries

Create

SQL

```
INSERT INTO expenses
VALUES (111,1,1,450,'2025-06-11');
```

Read

SQL

```
SELECT * FROM expenses;
```

Update

SQL

```
UPDATE expenses
```

```
SET amount = 600
```

```
WHERE expense_id = 111;
```

Delete

SQL

```
DELETE FROM expenses
```

```
WHERE expense_id = 111;
```

Stored Procedure for Monthly User Expense

MySQL

SQL

```
DELIMITER //
```

```
CREATE PROCEDURE MonthlyExpense(IN uid INT)
```

```
BEGIN
```

```
    SELECT
```

```
        user_id,
```

```
        MONTH(expense_date) AS month_no,
```



```
SUM(amount) AS total_expense
FROM expenses
WHERE user_id = uid
GROUP BY user_id, MONTH(expense_date);
END //
```

DELIMITER ;

Execute

SQL

CALL MonthlyExpense(1);

COMMIT and ROLLBACK Example

SQL

START TRANSACTION;

INSERT INTO expenses

VALUES (112,2,1,1000,'2025-06-12');

COMMIT;

Rollback Example

SQL

```
START TRANSACTION;
```

```
INSERT INTO expenses
```

```
VALUES (113,2,1,5000,'2025-06-12');
```

```
ROLLBACK;
```